



POLITECNICO DI TORINO

Corso di Laurea in Cybersecurity

Tesi di Laurea

# Iterative Approach for Network Security Automation

## **Relatori**

prof. Fulvio Valenza  
prof. Daniele Brighenti  
prof. Francesco Pizzato

## **Candidato**

Filippo DEL MINISTRO

ANNO ACCADEMICO 2025-2026



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

# Index

|          |                                     |           |
|----------|-------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                 | <b>5</b>  |
| 1.1      | Context . . . . .                   | 5         |
| 1.2      | VEREFOO . . . . .                   | 5         |
| 1.2.1    | Overview . . . . .                  | 5         |
| 1.2.2    | React-VEREFOO . . . . .             | 5         |
| <b>2</b> | <b>Iterative React-VEREFOO</b>      | <b>7</b>  |
| 2.1      | Idea of the Solution . . . . .      | 7         |
| 2.2      | Implementation . . . . .            | 7         |
| 2.3      | Example Run . . . . .               | 7         |
| <b>3</b> | <b>Experimental Evaluation</b>      | <b>8</b>  |
| 3.1      | Test Presentation . . . . .         | 8         |
| 3.2      | Test Configurations . . . . .       | 8         |
| 3.3      | Results . . . . .                   | 8         |
| 3.4      | Comparison and Discussion . . . . . | 8         |
| <b>4</b> | <b>Conclusions</b>                  | <b>9</b>  |
| <b>5</b> | <b>Executive Summary</b>            | <b>10</b> |

# Capitolo 1

## Introduction

### 1.1 Context

Modern enterprise networks are characterized by growing complexity: hundreds of interconnected nodes, heterogeneous services, and security policies that must be consistently enforced across distributed infrastructure. Packet-filtering firewalls represent the primary mechanism for enforcing such policies, selectively allowing or blocking traffic flows based on predefined rules. However, as networks scale in size and dynamism, manually configuring firewalls becomes error-prone, time-consuming, and increasingly difficult to reason about in terms of formal correctness. An automated approach is therefore necessary — one that can guarantee both the correctness of the generated configuration and its optimality with respect to resource usage. It is in this context that VEREFOO was developed: a tool for the automated, formally correct allocation of firewalls within a network topology and the automatic generation of their filtering rules.

### 1.2 VEREFOO

#### 1.2.1 Overview

VEREFOO (VERification REasoning on Firewall Optimization) is a tool designed to automate the allocation and configuration of packet-filtering firewalls in a network. It takes as input a network topology, a set of Network Security Requirements (NSRs) and a Firewall Allocation Schema (FAS), which defines the candidate positions in the topology where firewalls may be placed. From these inputs, VEREFOO produces two outputs: an updated topology indicating which nodes have been allocated as active firewalls, and a vendor-independent configuration for each firewall specifying the filtering rules to be enforced. The tool guarantees three fundamental properties: *automation*, as the entire allocation and configuration process requires no manual intervention; *formal correctness*, as the generated configuration is proven to satisfy all specified NSRs; and *optimality*, as the number of allocated firewalls and generated rules is minimised. These guarantees are achieved by encoding the problem as a MaxSMT instance and solving it with the Z3 solver. The resulting configuration can further be translated into vendor-specific formats.

#### 1.2.2 React-VEREFOO

In operational environments, network security policies are not static: NSRs evolve over time as new services are deployed, topology changes occur, or threat models are updated. A fundamental limitation of the monolithic approach used by VEREFOO is that any change to the NSR set triggers a full recomputation of the firewall configuration from scratch. React-VEREFOO addresses this by offering a more efficient approach in the dynamic context described above, avoiding the

need to discard a valid existing configuration when only a small portion of the requirements has changed.

React-VEREFOO takes as input both the existing firewall configuration and the updated set of security requirements, and computes only the minimal changes needed to bring the network into compliance. Rather than reconsidering the entire problem, it identifies which requirements are new, which have been removed, and which remain unchanged. Only the newly added requirements drive the reconfiguration: the approach determines which parts of the network are affected and restricts the solver to those areas, leaving the rest of the configuration untouched. The result is an updated firewall allocation and rule set that satisfies all current requirements, with the same guarantees of correctness and optimality as VEREFOO.

Experimental results show that React-VEREFOO is significantly more efficient than vanilla VEREFOO when the proportion of modified NSRs is small, as the reduced problem size leads to much shorter solving times. However, when a large fraction of the requirements changes, the benefits is reduced and React-VEREFOO's performance becomes comparable to that of the monolithic approach. This observation motivates the approach investigated in this thesis: by pushing the incremental logic to its extreme - introducing only one new NSR per iteration - the proportion of modified requirements is always minimised, potentially achieving consistent speedups regardless of the total number of requirements.

## Capitolo 2

# Iterative React-VEREFOO

### 2.1 Idea of the Solution

As discussed in the introduction, React-VEREFOO is most effective when the number of additional requirements is small relative to the total. The key idea of the approach is to push that observation to its logical extreme: instead of processing all new NSRs at once, the proposed approach introduces them one at a time, turning the configuration problem into a sequence of minimal reconfiguration steps.

The process works as follows. At each iteration, **the current firewall configuration is treated as the initial state and one single new NSR is added as the target**. React-VEREFOO is then invoked to compute the minimal update needed to satisfy that one additional requirement. The output of each iteration, meaning the updated allocation scheme and the associated filtering rules becomes the initial state for the next one. This continues until all NSRs have been incorporated.

Formally, if the set of additional NSRs to be processed is  $\{A, B, C, \dots\}$  and  $P_0$  denotes the initial set of properties already enforced in the network, the sequence of iterations is:

1. initial:  $P_0$ , additional:  $\{A\}$
2. initial:  $\{P_0, A\}$ , additional:  $\{B\}$
3. initial:  $\{P_0, A, B\}$ , additional:  $\{C\}$
4. ...

Since each invocation of React-VEREFOO deals with exactly one added requirement, the portion of the network that must be reconsidered at each step is minimised by construction. This keeps each individual MaxSMT subproblem small, regardless of the total number of NSRs, with the expectation of reducing overall computation time. The number of iteration steps is equal to the number of new NSRs, which is a linear factor, but the reduced complexity of each step is expected to more than compensate for that.

### 2.2 Implementation

### 2.3 Example Run

## Capitolo 3

# Experimental Evaluation

### 3.1 Test Presentation

### 3.2 Test Configurations

### 3.3 Results

### 3.4 Comparison and Discussion



## Capitolo 4

## Conclusions

## Capitolo 5

# Executive Summary